Memory Layer

Purpose

Memory Layer is responsible for preserving the continuity of the digital character.

Its purpose is not storing data.

Its purpose is allowing Shio to remember.

The Runtime does not preserve memory for the benefit of the language model.

It preserves memory for the benefit of the character.

Memory allows conversations, experiences and relationships to extend beyond a single interaction.

Without memory, every conversation begins with a new character.

With memory, the same character continues living.

Memory Is Not Storage

Storage is an implementation detail.

Memory is an architectural concept.

SQLite is not Memory.

PostgreSQL is not Memory.

A vector database is not Memory.

They are storage providers.

Memory Layer defines:

- what should be remembered;
- why it should be remembered;
- when it should be remembered;
- how remembered information influences future behavior.

Storage simply persists it.

Responsibilities

Memory Layer owns:

- long-term memories;
- episodic memories;
- conversation summaries;
- remembered user facts;
- remembered character experiences;
- memory retrieval;
- memory consolidation;
- forgetting policies.

Memory Layer never owns:

- dialogue generation;
- emotions;
- rendering;
- speech;
- external integrations.

Why Memory Exists

Memory serves one purpose:

Maintaining continuity.

For example:

Without memory:

```text id="mem01" User

↓

Conversation

↓

Application closes

↓

Everything disappears
```

````
With memory:

```text id="mem02"
User

↓

Conversation

↓

Memory

↓

Next conversation

↓

Continuation
````

The user interacts with the same character rather than a newly created one.

Memory Is For Shio

The architecture intentionally avoids treating memory as a knowledge base.

Instead, memory represents the accumulated experiences of the character.

Examples include:

"I met this user before."

"We worked on the architecture yesterday."

"They prefer concise answers."

"I promised to continue this project."

These memories belong to Shio.

They are not implementation artifacts.

````

Memory Categories

Different memories serve different purposes.

Episodic Memory

Individual experiences.

Examples:

- conversations;
- completed projects;
- important events;
- memorable moments.

Semantic Memory

Persistent knowledge about users and the world.

Examples:

- preferred language;
- recurring interests;
- established relationships;
- long-term facts.

Working Memory

Short-lived information required for the current interaction.

Examples:

- active conversation;
- temporary context;
- unfinished reasoning;
- pending actions.

Working Memory disappears naturally after the interaction ends.

Procedural Memory (Future)

Knowledge about how the character behaves.

Examples:

- recurring habits;
- favorite greetings;
- characteristic jokes;
- communication style.

Procedural Memory strengthens personality rather than storing factual information.

Memory Retrieval

Memory should never be loaded indiscriminately.

Instead, Memory Layer retrieves only the information relevant to the current interaction.

The objective is contextual continuity rather than maximum recall.

Good memory retrieves less, but retrieves the right information.

Memory Consolidation

Not every conversation deserves permanent storage.

After meaningful interactions, Memory Layer may consolidate temporary information into long-term memory.

Examples include:

- important decisions;
- recurring preferences;
- promises;
- relationships;
- memorable experiences.

Routine conversation should naturally fade over time.

Forgetting

Forgetting is considered a feature rather than a limitation.

Without forgetting, memory quality gradually deteriorates.

Memory Layer therefore supports controlled forgetting.

Examples include:

- expired temporary context;
- obsolete preferences;
- insignificant conversations;
- duplicated information.

The objective is preserving meaningful continuity rather than maximizing stored data.

Personality And Memory

Personality answers:

> "Who is Shio?"

Memory answers:

> "What has Shio experienced?"

Neither replaces the other.

Together they create long-term identity.

Experience And Memory

Experience Layer generates experiences.

Memory Layer decides whether those experiences deserve preservation.

For example:

```text id="mem03" Conversation

↓

Experience

↓

Memory Evaluation

↓

Stored
```

or

Discarded ```

This separation prevents every interaction from becoming permanent memory.

Storage Independence

Memory Layer never depends on a specific storage implementation.

Possible implementations include:

- in-memory storage;
- SQLite;
- PostgreSQL;
- vector databases;
- cloud storage;
- future providers.

Changing storage should never change Runtime behavior.

Memory Interface

Other Runtime systems never communicate directly with storage.

They communicate only with Memory Layer.

This abstraction allows the storage implementation to evolve independently from the Runtime.

Future Evolution

Memory Layer is expected to become one of the central Runtime systems.

Future capabilities may include:

- relationship memory;
- emotional memory;
- memory confidence;
- memory decay;
- autobiographical memory;
- long-term character development.

Regardless of future implementation, the architectural responsibility remains unchanged:

Memory Layer preserves the continuity of the character across time.